

Анатомия **Single Page Apps** и архитектура React

Природа и архитектура SPA

Эволюция веба от многостраничных монолитов к реактивным клиентским интерфейсам.

#01

Архитектура

Ограничения классических MPA

В традиционной многостраничной модели каждый клик по ссылке отправляет запрос на сервер. Браузер сбрасывает страницу, заново парсит HTML, блокируя интерфейс и повторно загружая одни и те же стили и скрипты.

#02

Концепция

Анатомия Single Page App

SPA единожды отдает базовый HTML-каркас и JS-бандл. Все дальнейшие экраны, формы и переходы отрисовываются локально на клиенте с подтягиванием чистых данных в формате JSON через API.

#03

Маршрутизация

Клиентский Client-Side Routing

Использование `history.pushState()` и события `popstate` позволяет менять URL в строке браузера без перезагрузки документа, синхронизируя адрес с активным деревом компонентов.

```
window.history.pushState({},  
'', '/dashboard');
```

#04

Инженерия

Плюсы и компромиссы SPA

SPA гарантирует бесшовный app-like UX и низкую нагрузку на сервер при переходах. Однако требует контроля за размером JS-бандла, утечками памяти в долгоживущих сессиях и оптимизации первого рендера.

Фундамент библиотеки React

Декларативное описание пользовательских интерфейсов и компонентная модель.

#05

Парадигма

Декларативный подход

Вместо ручного поиска узлов через `querySelector` и вызовов `appendChild`, разработчик описывает конечное состояние UI: «каким интерфейс должен быть при данных X».

#06

Структура

Компонентная декомпозиция

Интерфейс собирается из изолированных, переиспользуемых функций. Каждый компонент инкапсулирует свою верстку, стили и локальную бизнес-логику, собираясь в единое иерархическое дерево.

#07

Data Flow

Однонаправленный поток (Props)

Данные передаются строго сверху вниз — от родительских компонентов к дочерним через неизменяемые параметры (`props`). Дочерний компонент не может напрямую модифицировать входные пропсы.

#08

Синтаксис

JSX и транспилиция

JSX — синтаксический сахар, объединяющий HTML-подобную разметку с мощностью JavaScript. Компиляторы (Vite, Babel, SWC) преобразуют JSX в функции создания виртуальных объектов.

```
const el =  
<h1>Привет</h1>; //  
React.createElement
```

Virtual DOM и Reconciliation

Механизмы высокопроизводительного обновления реального дерева документа.

#09

Рендеринг

Сущность Virtual DOM

Virtual DOM — это легковесное дерево JavaScript-объектов в оперативной памяти браузера. Создание и мутация таких объектов происходит в разы быстрее прямого обращения к тяжелому API DOM браузера.

#10

Алгоритмы

Алгоритм Diffing

React сравнивает предыдущий и новый снимок виртуального дерева по эвристическому алгоритму со сложностью $O(n)$, определяя, какие именно свойства и текстовые ноды изменились.

#11

Оптимизация

Важность атрибута Key

Специальный prop key сообщает механизму согласования стабильную идентичность элементов в списках. Не используйте индексы массива, если порядок элементов может динамически меняться.

```
<li key={item.uuid}>
  {item.title}</li>
```

#12

Ядро React

Движок React Fiber

Архитектура Fiber разбивает процесс рендеринга на мелкие кванты времени (units of work). Это позволяет приостанавливать, отменять и приоритизировать рендеринг без фризов пользовательского ввода.

Система хуков и реактивность

Управление локальным состоянием, сайд-эффектами и жизненным циклом.

#13

State

useState: Иммутабельность

Состояние нельзя мутировать напрямую. Для запуска цикла перерисовки вызывается сеттер-функция, принимающая новое значение или функцию-преобразователь:

```
setCount(prev => prev + 1).
```

#14

Сайд-эффекты

useEffect и Cleanup

Хук для сетевых запросов, таймеров и подписок. Функция очистки (cleanup) предотвращает утечки памяти при размонтировании компонента или перед повторным запуском эффекта.

```
useEffect(() => { return  
  () => cleanup(); },  
  [dep]);
```

#15

Перформанс

useMemo и useCallback

Мемоизируют результаты тяжелых вычислений (useMemo) и сохраняют ссылочную идентичность функций (useCallback), предотвращая бесполезные рендеры дочерних компонентов.

#16

Ссылки и Контекст

useRef и useContext

useRef хранит мутабельное значение, не вызывающее ререндер (и дает доступ к DOM-узлу), а useContext передает глобальные данные сквозь дерево без ручного prop-drilling.

Современный продакшен-стек

Масштабирование, серверный рендеринг и оптимизация приложений.

#17

Глобальный стейт

Менеджеры состояния

Разделение стейта на серверный (кеш сетевых запросов через TanStack Query / RTK Query) и локальный UI-стейт (легкие решения вроде Zustand). Это устраняет бойлерплейт и ускоряет UI.

#18

Оптимизация

Code Splitting и Suspense

Разбиение общего бандла на маршрутные чанки с помощью `React.lazy`. Пользователь загружает код конкретной страницы только тогда, когда действительно переходит на нее.

```
const Profile = lazy(() =>
import('./Profile'));
```

#19

Гибридный рендер

SSR, SSG и React Server Components

Современные фреймворки рендерят компоненты на сервере без отправки лишнего JS клиенту (RSC), отдавая чистый HTML с последующей быстрой гидратацией (Selective Hydration).

#20

Профилирование

React DevTools & Web Vitals

Контроль ключевых метрик: LCP (скорость загрузки контента), INP (отзывчивость на действия пользователя) и CLS (стабильность верстки) с использованием React Profiler.